Tracking the Lifecycle of Kubernetes Configuration Smells Across Versions: Preliminary Evidence (Milestone 3)

Group 4
ENGR 5570G Software Maintenance and Evolution, Ontario Tech University
Rohan Muslekar (101006689), Ankit Suman (101041764), Yasaminzahra Valitabar (101001242),
Bayan Sayyadifar (101035809), Seyedeh Rojan Razavi (101001141)

***Abstract*: We follow the configuration smells that kube-linter and kube-score report in the Kubernetes manifests of 7 open-source projects across every minor release since late 2022 (123 versions). Smells are sticky: 82.3% of the 821 instances (counted per tool) present at a project's first sampled release are still present at its last, and none was removed and later reintroduced. Of 147 removals, 72.1% happened because the object was deleted, but two commits deleting 8 objects account for all of them; per object and category, deletions and real fixes are even (24 and 25). Tracing every fix and every introduction on an existing object to its commit shows that 24 of the 32 fixes come from four commits, mostly on security smells and at least 1.4 years (median) after the smell was first seen, and that most new smells on existing objects are side effects of changes made for another reason. Rendering every kustomization, our strongest validity threat, keeps survival, the deletion share and the fix sources but adds deleted flux2 objects, so that per object deletions outnumber fixes (36 to 24); sampling every release instead of five changes almost nothing. A blind LLM rater and an AI-assisted team rating agree with the category map on 59 and 58 of 62 checks ($\kappa = 0.93, 0.91$).**

I. Research Questions and Progress

Questions. RQ1 (prevalence, supporting): which smell categories occur in released Kubernetes manifests, and in how many instances, checks and systems? RQ2 (lifecycle, primary): once a smell instance appears, does it persist, disappear, or reappear in later releases, and when it disappears, is the smell fixed or the object removed? RQ3 (explanation, primary): which changes cause smells to appear and disappear? The scope is the manifests a project ships in its repository, analyzed at released tags. Table I links each question to the evidence that answers it.

Changes since Milestone 2. Each change answers a weakness that Milestone 2 reported.

1) **Every minor release instead of five.** Five tags over four years can hide a smell that is removed and re-added between two sampled tags, so zero reappearances was only a lower bound. We now sample every minor release, 123 versions.
2) **Rendering check.** Milestone 2 linted kustomize bases and patches as stored, and 84 of its removals were flux2 patch fragments, so the deletion finding rested on unrendered input. We rendered every kustomization in the study with `kubectl kustomize` at every tag and compared (Section IV).
3) **Category validation.** The check-to-category map was written by one author; the procedure for two human raters is in place; a blind automated rater and an AI-assisted rater 1 have rated all 62 checks.
4) **Rename detection.** A renamed object used to count as one removal and one introduction; tracking now flags probable renames.
5) **Commit tracing** grows from 4 spot-checked fixes to every fix, every introduction on an existing object and the commits behind all deletions (RQ3).
6) **RQ4 (image CVEs) dropped**, as feedback put the weight on the lifecycle.

II. Methods Executed

Systems and versions. Table II lists the 7 systems. A candidate is included when it ships plain Kubernetes YAML in-tree, had commits in the last six months and has at least 500 stars, and has at least five release tags over at least 12 months; 4 failed and are recorded with the reason in `data/repos.json`. We take the latest patch of every minor release first published since 2022-10-22 whose tagged commit is dated by 2026-10-22, and keep every Milestone 2 tag (`python3 -m kst.select_tags work/<repo> 2022-10-22 --all --until 2026-10-22 --keep <M2 tags>`). The list is frozen in `data/repos.json`: a patch can be tagged weeks after its commit, so on clones fetched on 2026-11-07 the command picks a later patch for 4

TABLE I
Research questions and the evidence that answers them

RQ	Evidence in this report	Status
RQ1	Instances per category and tool (Table III), per system (Table II), most frequent checks (Table IV)	answered
RQ2	Transitions (Table III), survival per system (Fig. 1) and per category (Table V), removal causes, smell age, reappearance over all minor releases	answered
RQ3	77 transitions traced to commits (Table VI): every fix and every introduction on an existing object, with cases; the two commits behind all deletions	answered

of the 123 tags. `data/versions.csv` records the commit of each version; all 123 are valid. Tags are compared in version order. A tag is the latest patch on its minor's release branch, so a step can carry backports and a tag can be dated years after its minor shipped; ages, Table II and Fig. 1 therefore date each tag by its minor's x.y.0 (`data/minor_dates.csv`).

Pipeline. For each tag, `scripts/collect.sh` checks out the tag, keeps the plain Kubernetes YAML under the manifest path and logs every skipped file with its reason (388 kustomization files over all tags, 382 of them in flux2). It records the objects defined at the tag and runs kube-linter v0.8.3 [1] (all built-in checks, pinned configuration) and kube-score v1.20.0 [2]. A tag whose analyzer output is empty or not JSON is invalid and left out, so a crash does not look like removed smells. The 62 observed checks map to four categories seeded from Kubernetes misconfiguration and configuration-smell work [3], [4], [5], or to *other* (style and opinionated defaults, excluded). An instance is keyed by (project, tool, kind, namespace, name, check); each pair of consecutive tags labels it persisted, introduced, removed or reappeared, and the object inventory separates removals where the object was deleted from fixes.

Executed since Milestone 2.

- **Dense tags** (PR #10): the tag selection above; `kst.subset` cuts any run back to the five Milestone 2 tags for comparison.
- **Rendering check** (PR #15): a new script, `scripts/render_check.py`, renders every kustomization that no other kustomization lists (15 in flux2 at the first tag, one in microservices-demo, the only projects that use kustomize), drops the plain files they use, and runs the same inventory, analyzers and tracking at all 22 of their tags. Ankit's change (PR #14) keeps rendered paths machine-independent.
- **Category validation** (PR #11): `kst.category_validation prepare` writes blank sheets of all 62 checks with the definitions but not the map; `calculate` gives agreement and Cohen's kappa. An LLM (Claude Opus 5.5) rated one sheet blind (PR #19), from the definitions and the analyzers' documentation only, never the repository or the map. Rater 1's sheet (PR #21) says it is AI-assisted.
- **Rename detection** (PR #12): an object that disappears and one that appears between consecutive tags are flagged as a probable rename when they share the kind and the same non-empty set of (tool, check); histories are not merged.
- **Commit tracing** (PR #13, PR #17): for each fix and each introduction on an existing object, `git log <from>..<to> -- <file>` over its two consecutive tags finds the commit; we read its diff and record the commit, the configuration change, an evidence link and a cause in `data/rq3_investigations.csv`. Causes: *real fix* (the diff changes the setting the check reads), *side effect* (a commit made for another reason adds the smell), *new object*, *analyzer artifact* (no configuration change), *object deleted*. Bayan coded the 35 Milestone 2 fixes and Rohan the rest, one coder per row. `scripts/check_rq3.py` confirms that every traced commit lies between its two tags; `scripts/deletion_commits.py` lists the commits that delete manifest files in each step with a deletion.
- **Reproduction**: CI reruns 118 tests and the offline analysis on every change and fails if `out/` differs. A fresh clone at the data merge (PR #16) recollected all 123 tags in 17 min with byte-identical `out/` and `data/versions.csv`; raw files differed only in kube-linter's timestamp and kube-score's check order.

Deviations from plan. Rendering inside the pipeline (`kst/render.py`, `RENDER=1`, now merged in PR #20) was not ready for the integration run, so the main results use stored manifests and the rendering check, which covers all 388 skipped kustomization files, gives the rendered results beside them. Rater 1's sheet came back AI-assisted and rater 2's has not; human agreement moves to Milestone 4.

Rerunning. From https://github.com/Rohan-Muslekar/kube-smell-tracker: `./tools/install.sh && make all` recollects every tag and rebuilds `raw/` and `out/`; `python3 -m kst` reruns the analysis offline; `scripts/minor_dates.py`, `survival.py`, `check_rq3.py` and `deletion_commits.py` rebuild the release dates, Fig. 1 and the trace checks; `scripts/render_check.py` at commit `1e314b0` (PR #15) reruns the rendering check. The Milestone 2 data is kept at git tag `milestone-2-data`. The README lists the environment (Fedora 44, Go 1.26.9, Python 3.14.8 with PyYAML 6.0.3 for rendering, kubectl 1.36.2 with kustomize 5.8.1). No LLM is used except in the category ratings.

III. Preliminary Findings

RQ1: prevalence. Over the 123 versions the analyzers report 862 smell instances (483 kube-linter, 379 kube-score; 276 object–category pairs when both tools are merged). Resource and reliability smells are the most common (420, 48.7%), mostly missing probes, limits and replica settings, followed by security (261, 30.3%), image (76) and hygiene (105). Per system the count ranges from 56 (Tekton pipeline) to 239 (istio's bookinfo samples, Table II), roughly with the number of workloads shipped. The smells concentrate on a few checks (Table IV): eight account for 44.0% of all instances, and six of them occur in at least six of the seven systems. The exception, `use-namespace` (3 systems), fires on manifests that leave the namespace to the installer.

RQ2: lifecycle. 676 of 821 first-tag instances (82.3%) are present at the last tag, and 676 of all 862 instances (78.4%) are present at every sampled tag. Across 123 versions, 0 instances reappeared. Of 147 removals, 106 (72.1%) are object deletions and 41 keep the object; 9 of those are a kube-score artifact (it stops comparing probes once they use gRPC), leaving 32 real fixes (25 once both tools are merged). The 106 deletions

TABLE II

STUDY SYSTEMS AND PER-SYSTEM RESULTS (SMELLS EXCLUDING *other*). SURV.: SHARE OF FIRST-TAG INSTANCES PRESENT AT THE LAST TAG. PARENTHESES: REMOVALS WITH THE OBJECT DELETED; INTRODUCTIONS WITH A NEW OBJECT. YEARS: THE FIRST RELEASE OF EACH TAG'S MINOR (X.Y.0), AS IN FIG. 1.

System	Manifest path	Tags	First–last tag (years)	Inst.	Surv.	Removed	Introd.
microservices-demo	`kubernetes-manifests`	6	v0.5.2–v0.10.7 (2022–2024)	182	89.0%	20 (0)	1 (0)
kube-prometheus	`manifests`	8	v0.12.0–v0.19.0 (2023–2026)	80	91.2%	7 (0)	0 (0)
argo-cd	`manifests/install.yaml`	16	v2.5.22–v3.5.6 (2022–2026)	127	98.2%	2 (0)	13 (1)
flux2	`manifests`	16	v0.36.0–v2.9.6 (2022–2026)	97	11.6%	84 (84)	2 (2)
istio	`samples/bookinfo/platform/kube`	16	1.16.7–1.31.2 (2022–2026)	239	100.0%	0 (0)	10 (0)
serving	`config/core`	15	v0.36.4–v0.50.0 (2023–2026)	81	65.4%	28 (22)	0 (0)
pipeline	`config`	46	v0.41.3–v1.17.0 (2022–2026)	56	90.2%	6 (0)	15 (13)

TABLE III

SMELL INSTANCES AND TRANSITIONS PER CATEGORY AND TOOL (KL = KUBE-LINTER, KS = KUBE-SCORE). BASE: PRESENT AT THE FIRST TAG; PERSIST: TAG-TO-TAG STEPS AN INSTANCE STAYS. PARENTHESES: INTRODUCTIONS WITH A NEW OBJECT; REMOVALS WITH THE OBJECT DELETED.

Category	Tool	Inst.	Base	Persist	Introd.	Removed	Reapp.
Security & privilege	KL	147	126	1996	21 (5)	39 (22)	0
Security & privilege	KS	114	111	1427	3 (2)	25 (16)	0
Resource & reliability	KL	216	212	3009	4 (4)	28 (24)	0
Resource & reliability	KS	204	199	2483	5 (3)	33 (22)	0
Image & supply chain	KL	25	24	334	1 (1)	6 (6)	0
Image & supply chain	KS	51	48	636	3 (1)	10 (10)	0
Hygiene	KL	95	91	1007	4 (0)	4 (4)	0
Hygiene	KS	10	10	112	0 (0)	2 (2)	0
Total		862	821	11004	41 (16)	147 (106)	0

TABLE IV

THE EIGHT MOST FREQUENT CHECKS. SYS.: SYSTEMS WHERE THE CHECK OCCURS; SURV.: SHARE OF ITS FIRST-TAG INSTANCES PRESENT AT THE LAST TAG.

Check	Tool	Inst.	Sys.	Surv.
`use-namespace`	KL	63	3	100.0%
`container-ephemeral-storage-request-and-limit`	KS	51	7	88.0%
`minimum-three-replicas`	KL	48	7	91.5%
`container-security-context-user-group-id`	KS	48	7	77.1%
`no-rolling-update-strategy`	KL	47	7	91.3%
`pod-networkpolicy`	KS	45	6	86.0%
`deployment-replicas`	KS	39	6	97.3%
`non-isolated-pod`	KL	38	5	83.8%

TABLE V

LIFECYCLE PER CATEGORY, BOTH TOOLS. FIRST: INSTANCES PRESENT AT THE FIRST TAG. REAL FIXES EXCLUDE THE ANALYZER ARTIFACT. PARENTHESES AS IN TABLE III.

Category	First	Surv.	Removed	Real fixes	Introd.
Security & privilege	237	73.8%	64 (38)	26	24 (7)
Resource & reliability	411	85.2%	61 (46)	6	9 (7)
Image & supply chain	72	77.8%	16 (16)	0	4 (2)
Hygiene	101	94.1%	6 (6)	0	4 (0)

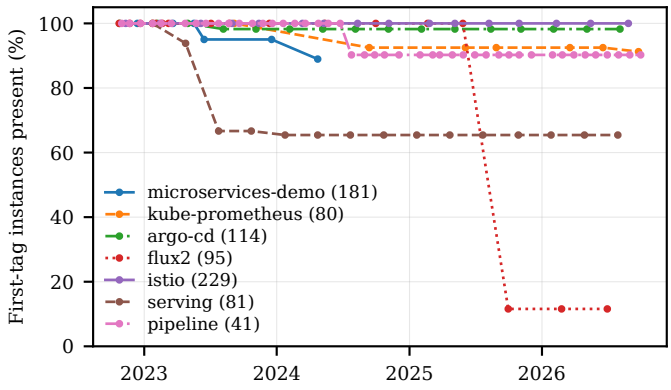


Fig. 1. Share of first-tag smell instances still present at each later release, per system (first-tag instances in parentheses), plotted at the date of the release's minor (x.y.0).

sit on only 8 objects and the real fixes on 25. The variation is large (Fig. 1): argo-cd and istio keep 98–100% of their first-tag smells and three more systems about 90%, while Knative serving keeps 65% and flux2 12%, losing most at once when it deletes a directory. Without flux2 only 34.9% of removals are deletions, so the deletion share is a property of two systems, not of all.

Survival also differs by category (Table V). Security smells survive least (73.8%) because 26 of the 32 real fixes remove security smells; resource smells get 6, and no image or hygiene smell was ever fixed, only deleted with its object. Hygiene smells survive most (94.1%). A chi-square test of independence between category and survival over the 821 first-tag instances is significant ($\chi^2(3) = 24.6$, $p < 0.001$) but the effect is small (Cramér's $V = 0.17$). It holds without flux2 ($\chi^2(3) = 17.3$, $p < 0.001$). With the 268 object–category pairs as the unit it depends on the rule: if a pair survives when any of its instances does, $\chi^2(3) = 6.7$, $p = 0.08$; if only when all do, $\chi^2(3) = 30.6$, $p < 0.001$ ($V = 0.34$). Smells on one object tend to go together, so instances are not independent, and we treat the category difference as suggestive. Fixes are also slow: a real fix comes a median of 1.4 years after the first release that shows the smell (0.2–3.7 years; 8 of 32 within a year), or 1.3 years by the fix commit's own date. Both are lower bounds: 30 of the 32 smells were already present at the first sampled release. The 715 instances present at the last tag have been present for a median of 3.8 years.

RQ3: what causes transitions. We traced 77 transitions to commits (`data/rq3_investigations.csv`): all 41 fixes, all 25 introductions on objects that already existed and 2 on new objects, plus 4 deletions and 5 persisting smells from the spot-check. 32 fixes are deliberate configuration changes and 9 the analyzer artifact. They come from only 12 commits

TABLE VI
COMMITS BEHIND THE 32 REAL FIXES, PER SYSTEM

System	Commits	Fixes	What changed
microservices-demo	1	11	a ServiceAccount per service
kube-prometheus	7	7	`runAsGroup`, `runAsNonRoot`, no RBAC wildcards, one component each
pipeline	1	6	read-only root filesystem
serving	2	6	probes on the controller and domain-mapping; host anti-affinity
argo-cd	1	2	read-only root filesystem on Redis

(Table VI), and four of them give 24; one microservices-demo commit (20585fa, a refactoring of its kustomize components) gives every service its own ServiceAccount and removes 11 `default-service-account` smells at once. The other 8 commits remove one smell each, 7 of them in kube-prometheus. All 106 deletions fall in two tag steps with one deleting commit each: flux2 removed its credentials-sync integrations (9cad95d) and Knative serving dropped its separate domain-mapping controller and webhook when it consolidated its webhooks (fc166ac). Deletions are thus project decisions about what to ship, not cleanups of smells.

New smells on existing objects are mostly side effects of changes made for another reason, usually a fix: 23 of 25, and all 25 come from 6 commits. In argo-cd the fix for advisory GHSA-9766-5277-j5hr enabled Redis authentication: a password read from a secret in four workloads, a Role that reads secrets and an init container together add ten smells. In istio, `runAsUser` was removed from every bookinfo Deployment because OpenShift rejects fixed UIDs; the images now set the user, so the containers still run as non-root, but a manifest-only analyzer reports ten new `run-as-non-root` smells. In microservices-demo, pinning the init container's `busybox:latest` to a digest changed its default pull policy and added a kube-score smell; `latest-tag` stays, as the main container is untagged.

Analyzer overlap. On object kinds both tools analyze, they agree on 134 of 241 object–category pairs (55.6%), 78.5% without hygiene, echoing the disagreement reported between static analyzers [6]; we therefore give counts per tool and merged. They differ most on hygiene, where 73 of 83 pairs are kube-linter only (62 of them `use-namespace`, which kube-score does not check), and on image smells, where 21 of 46 are kube-score only (its pull-policy check).

IV. INTERPRETATION AND VALIDITY

What the evidence supports. Within these 7 projects, a smell present in a release is very likely still present years later, and most disappearing instances leave with their object rather than through a fix, although per object the two are even in the stored manifests. Fixes that do happen are deliberate, and most come from a few commits that harden security settings or add probes. We do not claim this for Helm-only projects or for live cluster state, and the deletion share holds for flux2 and Knative serving, not for each system.

TABLE VII
VALIDITY THREATS AND THE CHECKS ALREADY RUN

Threat	Check run	Result
Unrendered kustomize	all rendered, every tag (PR #15)	survival, deletion share hold; deleted objects 8→14; 11 false positives
Sparse sampling	every minor, 123 tags (PR #16)	no reappearance; 2 removals change cause
Analyzer change as fix	every fix traced (PR #13, PR #17)	9 artifact removals excluded
Rename as fix	rename detection (PR #12)	0 smells flagged
Crash as removal	output validation per tag	all 123 valid
Dependent instances	chi-square on pairs	$p = 0.08$ or $p < 0.001$ by pair rule
One coder per cause	codebook, evidence links, deletion script	second coder planned
Non-reproducible	fresh clone from GitHub	`out/` byte-identical
Release-branch tags	x.y.0 and fix-commit dating; range check	1.4 vs 1.3 years; all in range
Single-author map	LLM (PR #19); AI-assisted rater 1 (PR #21)	59, 58/62; human pending

TABLE VIII
RENDERING CHECK AT EVERY TAG OF THE TWO PROJECTS WITH KUSTOMIZATIONS (S: STORED, R: RENDERED). ALL: THE WHOLE STUDY WITH BOTH PROJECTS RENDERED.

	flux2		microserv.-demo		All	
	S	R	S	R	S	R
Smell instances	97	162	182	166	862	911
Surv. to last tag	11.6%	24.2%	89.0%	88.6%	82.3%	79.6%
Removed	84	116	20	19	147	178
Deleted share	100.0%	100.0%	0.0%	0.0%	72.1%	77.5%
Real fixes	0	0	11	10	32	31
Reappeared	0	0	0	0	0	0

Comparison with prior work. The persistence matches Rahman et al.'s Puppet security smells, where a hard-coded secret lived a median of 20 months [5]; our real fixes come at least a median of 1.4 years after the smell is first seen. Dockerfile smell counts slowly decline [7], but a count cannot tell a fix from a deleted file; Rosa et al. [8] read the fixing commits and found that developers mostly act on image size and build time, while our deliberate fixes are mostly security hardening. Helm-chart work followed outdated images and security risks across chart versions [9], not individual smell instances, which is what our tracking adds.

Unexpected results. None of the 20 real fixes made before a system's last tag (in 4 systems) was undone later, which suggests that a smell once fixed stays fixed, as fixes are deliberate changes. The kube-score probe artifact shows that an analyzer can make a smell disappear without any configuration change; we keep these out of the fix count. We also did not expect whole categories to go unfixed: no image or hygiene smell was fixed in four years. Most hygiene instances (63 of 105, `use-namespace`) and the 36 pull-policy instances concern choices a project leaves to whoever installs it, so maintainers have little reason to change them. This does not explain the other 40 image instances, untagged or `latest` images that are the project's own choice; they persist too.

TABLE IX
PLAN TO THE FINAL REPORT (PRESENTATIONS NOV 27 AND DEC 4, REPORT DEC 11)

Task	Owner	Due	Fallback
Rendering as the pipeline default (`kst/render.py`); trace the transitions it adds (47 introductions, not 41)	Rojan, Ankit; Rohan	Nov 20	the rendering check (Table VIII) as rendered results
Human category ratings, kappa, ledger: Rojan blind to earlier sheets, Ankit re-rates without AI	Ankit, Rojan; Yasaminzahra	Nov 18	one rating: a consistency check against the map
Case studies; a second coder recodes 20 traced rows	Bayan	Nov 24	the three cases in Section III; one coder as a threat
Presentation	all	Nov 27	this report's results; open items as work in progress
Recheck every number against `out/`; repeat the fresh clone	Rohan	Dec 4	numbers come from `make_tables.py`: rerun, not retyped
Final report and artifact	all	Dec 11	unfinished items become limitations

TABLE X
MEMBER CONTRIBUTIONS SINCE MILESTONE 2 (PRS IN THE PROJECT REPOSITORY)

Member	Completed work and linked artifacts
Rohan Muslekar	Dense tags and `kst.subset` (PR #10); rendering check (PR #15, kept open); integration run (PR #16); RQ3 tracing and survival figure (PR #17); LLM rating (PR #19); release dating, trace and deletion checks (PR #23); plan (PR #9); reviews on PR #11–PR #14, PR #20; this report
Ankit Suman	Rendered-input support in `analyze.sh` and `normalize.py` with tests (PR #14); AI-assisted rater 1 sheet (PR #21)
Yasaminzahra Valitabar	Category-validation sheets, agreement and ledger (PR #11); rename detection with tests and `kst.rename_report` (PR #12)
Bayan Sayyadifar	Commit tracing of the 35 untraced Milestone 2 fixes (27 real, 8 analyzer artifact), with evidence links (PR #13); case studies
Seyedeh Rojan Razavi	Kustomize renderer `kst/render.py`, `RENDER=1` and tests (PR #20)

Table VII lists each threat and the check already run.

Strongest threat: unrendered manifests. Linting kustomize bases and patches as stored can invent smells on fragments and miss objects that only exist after rendering, so we rendered every kustomization at every tag (Table VIII). Rendering raises flux2's smells from 97 to 162: its controller Deployments are remote resources that stored manifests never contained, and their smells mostly persist. The 11 hygiene smells on network policies and selectors disappear once kustomize adds the common labels, so they were false positives. All 116 rendered flux2 removals are still deletions, at v2.7.5. In microservices-demo, rendering only drops the `loadgenerator` Deployment, which the kustomization comments out (16 instances). With both rendered, survival is 79.6%, the deletion share 77.5%, 31 real fixes remain (23 from the same four commits) and nothing reappears. One finding changes: the credentials-sync Deployments, CronJobs and RoleBindings flux2 deleted now count, so deletions sit on 14 objects and per object outnumber fixes (36 to 24 pairs).

Check run: sparse sampling. Cutting the 123-version run back to the five Milestone 2 tags reproduces Milestone 2's outputs; `instances.csv` differs only by the new `probable_rename` column. The 88 extra tags add no instance and no reappearance and leave survival at 82.3%. Only two removals change cause: probes added to serving's `domain-mapping` Deployment before its deletion, which five tags saw only as a deletion.

Alternative explanation. Persistence need not mean neglect: istio's bookinfo samples are minimal on purpose, and a maintainer may accept a missing CPU limit. Smells go when a project decides to harden a component, so we read RQ2 as what happens to reported smells, not as a defect rate.

Category construct validity. The category map is one author's. The blind LLM rater and rater 1's AI-assisted sheet agree with it on 59 and 58 of 62 checks (Cohen's $\kappa = 0.93$, 0.91) and with each other on 61, so rater 1 is not independent of an automated rating. All differences are checks the map puts in *other*. Their categories give 962 and 1013 instances, 82.6% and 82.9% survival, 74.1% and 75.0% deleted and no reappearance: RQ2 and RQ3 do not change, while RQ1 shifts toward resource smells (53.8% and 56.2% of instances, from 48.7%).

Other threats. Seven projects and two tools limit generality. Totals add the two tools, so a setting both flag counts twice; merged, there are 276 pairs and 25 real fixes.

Implications. A manifest lifecycle study should render kustomize first and separate deletions from fixes: without the inventory the 106 deletions would count as fixes, over three times the 32 real ones. Analyzers can also create lifecycle events with no configuration change (the probe artifact, a user set in the image).

V. REMAINING WORK AND CONTRIBUTIONS

Scope reductions and plan. RQ4 (image CVEs) is dropped, Helm-only projects stay excluded and checkov is not added. Table IX gives each task an owner, a date and a fallback. The rendering check and the two ratings leave survival, the deletion share and the fix sources unchanged, so the plan holds if both analysis items fall back.

References

[1] "KubeLinter, v0.8.3," https://github.com/stackrox/kube-linter, StackRox.

[2] "kube-score, v1.20.0," https://github.com/zegl/kube-score.

[3] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, 2023.

[4] T. Sharma, M. Fragkoulis, and D. Spinellis, "Does your configuration code smell?" in *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, 2016, pp. 189–200.

[5] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, 2019, pp. 164–175.

[6] V. Lenarduzzi, F. Pecorelli, N. Saarimäki, S. Lujan, and F. Palomba, "A critical comparison on six static analysis tools: Detection, agreement, and precision," *Journal of Systems and Software*, vol. 198, p. 111575, 2023.

[7] K. Eng and A. Hindle, "Revisiting Dockerfiles in open source software over time," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, 2021, pp. 449–459.

[8] G. Rosa, F. Zappone, S. Scalabrino, and R. Oliveto, "Fixing Dockerfile smells: An empirical study," *Empirical Software Engineering*, vol. 29, p. 108, 2024.

[9] A. Zerouali, R. Opdebeeck, and C. De Roover, "Helm charts for Kubernetes applications: Evolution, outdatedness and security risks," in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023, pp. 523–533.